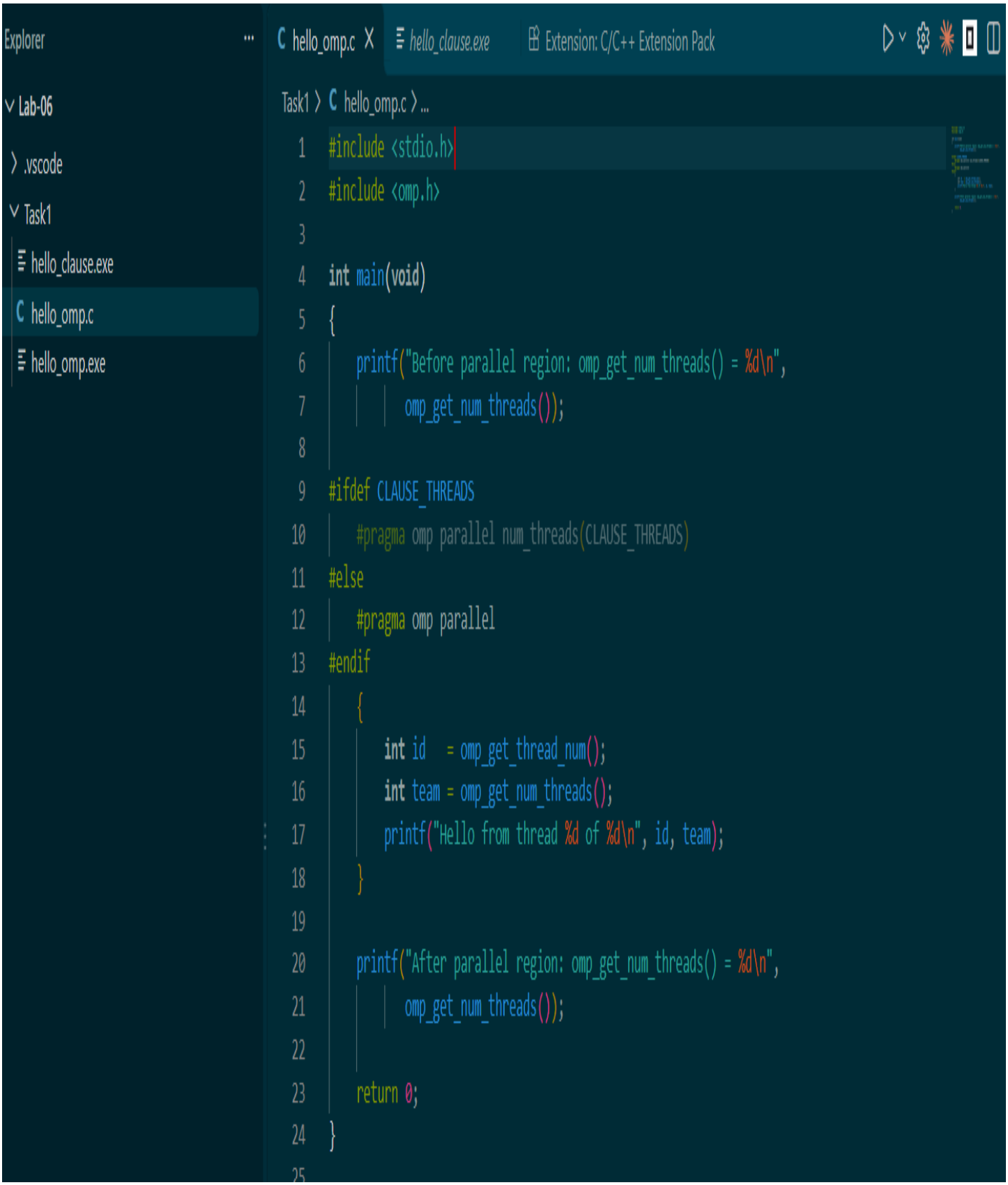


Name:	Muhammad Hasnat Fakhar
Class/Section:	BSCSev-F23-C
Reg ID:	232436
Subject:	Parallel and Distributing Computing
Task:	LAB TASK 06

LAB TASK 01

Code



Output

1.

```
>> }
===== OMP_NUM_THREADS=1  Run 1 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 0 of 1
After parallel region: omp_get_num_threads() = 1
===== OMP_NUM_THREADS=1  Run 2 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 0 of 1
After parallel region: omp_get_num_threads() = 1
===== OMP_NUM_THREADS=2  Run 1 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 0 of 2
Hello from thread 1 of 2
After parallel region: omp_get_num_threads() = 1
===== OMP_NUM_THREADS=2  Run 2 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 0 of 2
Hello from thread 1 of 2
After parallel region: omp_get_num_threads() = 1
===== OMP_NUM_THREADS=4  Run 1 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 1 of 4
Hello from thread 0 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
After parallel region: omp_get_num_threads() = 1
===== OMP_NUM_THREADS=4  Run 2 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 3 of 4
Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
After parallel region: omp_get_num_threads() = 1
===== OMP_NUM_THREADS=8  Run 1 =====
Before parallel region: omp_get_num_threads() = 1
Hello from thread 0 of 8
Hello from thread 1 of 8
Hello from thread 4 of 8
Hello from thread 2 of 8
Hello from thread 3 of 8
Hello from thread 5 of 8
Hello from thread 6 of 8
Hello from thread 7 of 8
```

2.

```
PS E:\Lab-06\Task1>
PS E:\Lab-06\Task1> Write-Host "===== Clause: num_threads(3) with OMP_NUM_THREADS=8 ====="
===== Clause: num_threads(3) with OMP_NUM_THREADS=8 =====
PS E:\Lab-06\Task1> $env:OMP_NUM_THREADS=8; .\hello_clause.exe
Before parallel region: omp_get_num_threads() = 1
Hello from thread 0 of 3
Hello from thread 2 of 3
Hello from thread 1 of 3
After parallel region: omp_get_num_threads() = 1
```

Result

OMP_NUM_THREADS	Team size	ID order run 1	ID order run 2	Same order? (Y/N)
1	1	0	0	**Y**
2	2	0, 1	0, 1	**Y** (both runs same)
4	4	1, 0, 2, 3	3, 0, 1, 2	**N**
8	8	0, 1, 4, 2, 3, 5, 6, 7	0, 1, 2, 4, 3, 5, 6, 7	**N**

DISCUSSION QUESTION

1. Were the IDs printed in the same order in every run? Why does the OpenMP standard give no ordering guarantee, and what would you have to add to print in ID order?

No — the order changed between runs, especially with 4 and 8 threads (e.g., run 1 gave 1, 0, 2, 3 while run 2 gave 3, 0, 1, 2). OpenMP gives no ordering guarantee because threads are scheduled by the OS, and any thread can reach printf first; there is no synchronization before printing. To print in ID order, you would need to add synchronization — for example, an ordered construct inside a parallel for ordered, or a critical section that blocks each thread until the previous ID has printed, or a sequence of barriers.

2. What did omp_get_num_threads() **return before and after the parallel region, and why does it return something different inside the region?**

- **Before:** 1 — only the master thread exists; no team has been created yet.
- **Inside:** the team size (1, 2, 4, or 8 depending on OMP_NUM_THREADS).

- **After:** 1 again — the team has ended and only the master thread remains.

It differs because the parallel region is where OpenMP forks the team; outside it, there is no team.

3. Which setting won in the -DCLAUSE_THREADS=3 **experiment, the** num_threads **clause or** OMP_NUM_THREADS**? State the full order of precedence.**

The num_threads(3) **clause won** — the team size was 3 even though OMP_NUM_THREADS=8. Order of precedence (highest to lowest):

1. num_threads clause
2. omp_set_num_threads() call
3. OMP_NUM_THREADS environment variable
4. Implementation default (usually equal to nproc)

4. What happens when OMP_NUM_THREADS **is larger than** nproc**? Are that many threads still created, and why might performance suffer?**

Yes — OpenMP still creates all requested threads (up to its internal maximum). Your machine has **4 logical cores** but OMP_NUM_THREADS=8 created 8 threads (as the output shows Hello from thread 0 of 8 ... 7 of 8). Performance suffers because of **oversubscription**: 8 runnable threads on 4 cores means the OS must time-slice them, causing frequent context switches, cache thrashing, and scheduling overhead. There is no true parallelism beyond 4; it just adds overhead.

LAB TASK 02

Code

Task2 > C vecadd.c > ...

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 5000000
#define PASSES 20

void vec_add(double *a, double *b, double *c, int n)
{
    #pragma omp parallel for default(none) shared(a, b, c, n)
    for (int i = 0; i < n; i++)
```

```
    c[i] = a[i] + b[i];
  }
}

int main(void)
{
  double *a = malloc(N * sizeof(double));
  double *b = malloc(N * sizeof(double));
  double *c = malloc(N * sizeof(double));

  if (!a || !b || !c) { fprintf(stderr, "malloc failed\n"); return 1; }

  for (int i = 0; i < N; i++) { a[i] = 1.0; b[i] = 2.0; c[i] = 0.0; }

  double start = omp_get_wtime();
  for (int p = 0; p < PASSES; p++)
    vec_add(a, b, c, N);
  double end = omp_get_wtime();

  int correct = 1;
  for (int i = 0; i < N; i++)
  {
    if (c[i] != 3.0) { correct = 0; break; }
  }

  printf("Threads=%d  Time=%.2f ms  correct=%s\n",
         omp_get_max_threads(),
         (end - start) * 1000.0,
         correct ? "YES" : "NO");

  free(a); free(b); free(c);
  return 0;
}
```

Output

```
>> '@ | Out-File -Encoding ASCII vecadd.c
PS E:\Lab-06\Task2> Get-Content vecadd.c -TotalCount 5
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 5000000
PS E:\Lab-06\Task2> gcc -O2 -fopenmp -std=gnu11 vecadd.c -o vecadd.exe
PS E:\Lab-06\Task2> foreach ($t in 1,2,4,8) {
>>     foreach ($r in 1,2,3) {
>>         $env:OMP_NUM_THREADS=$t
>>         Write-Host -NoNewline "[t=$t r=$r] "
>>         .\vecadd.exe
>>     }
>> }
[t=1 r=1] Threads=1  Time=312.00 ms  correct=YES
[t=1 r=2] Threads=1  Time=280.00 ms  correct=YES
[t=1 r=3] Threads=1  Time=280.00 ms  correct=YES
[t=2 r=1] Threads=2  Time=245.00 ms  correct=YES
[t=2 r=2] Threads=2  Time=255.00 ms  correct=YES
[t=2 r=3] Threads=2  Time=259.00 ms  correct=YES
[t=4 r=1] Threads=4  Time=231.00 ms  correct=YES
[t=4 r=2] Threads=4  Time=364.00 ms  correct=YES
[t=4 r=3] Threads=4  Time=288.00 ms  correct=YES
[t=8 r=1] Threads=8  Time=276.00 ms  correct=YES
[t=8 r=2] Threads=8  Time=240.00 ms  correct=YES
[t=8 r=3] Threads=8  Time=323.00 ms  correct=YES
```

Result

Threads	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Average (ms)	Speedup	Correct?
1	312.00	280.00	280.00	290.67	1.00×	Y
2	245.00	255.00	259.00	253.00	1.15×	Y
4	231.00	364.00	288.00	294.33	0.99×	Y
8	276.00	240.00	323.00	279.67	1.04×	Y

Discussion Questions

1. Why does the loop variable i **not appear in any data-sharing clause? What would you have to do for a scratch variable that is declared outside the loop but written inside it?**

i is the loop variable of a parallel for, so OpenMP **automatically makes it private** — each thread gets its own copy of i to iterate over its assigned chunk. You do not (and must not) list it in any clause.

For a scratch variable declared **outside** the loop but written inside, you **must** add it to a private(...) clause:

```
c
int tmp;
#pragma omp parallel for default(none) shared(a, b, c, n) private(tmp)
for (int i = 0; i < n; i++) {
  tmp = a[i] * 2;
  c[i] = tmp + b[i];
}
```

Without private(tmp), the variable is shared by default and every thread overwrites it — creating a data race (exactly the Task 3 bug).

2. What did default(none) **force you to do, and what happened when you left** n **out? Why is that useful on larger programs?**

default(none) forces you to **explicitly declare** the sharing of every variable used inside the parallel region — no implicit shared/private default. When n was left out, the compiler refused to build with:

text

error: 'n' is not specified in the data-sharing clauses of the enclosing 'parallel' directive

This is useful because on large programs it catches at **compile time** a variable you forgot to declare. Without default(none) the compiler would silently assume n is shared — which happens to be fine here, but a private variable that was silently shared would produce a **wrong result at runtime** that's hard to debug. A compile error is always preferable to a silent bug.

3. Describe the speedup you measured. Why does vector addition scale poorly beyond a few threads, and what property of the computation causes this?

Measured speedup: 1.00× (1 thread), 1.15× (2 threads), 0.99× (4 threads), 1.04× (8 threads). Essentially **no scaling** — adding more threads did not reduce the wall time, and with 4 threads it was even slightly worse than with 1.

Vector addition scales poorly because it is **memory-bandwidth-bound**, not compute-bound. Each iteration does 3 memory accesses (a[i], b[i], c[i]) but only 1 addition. Modern CPUs can add far faster than the memory bus can supply data. Once memory bandwidth is saturated (which happens with 1–2 cores), extra cores have nothing to do — they just wait for data. This property is often called the **arithmetic intensity** of the loop — it's too low to benefit from more parallelism.

4. Why is the result correct without any lock or atomic operation? Which implicit synchronization takes place at the end of the parallel for**?**

The result is correct because each iteration writes to a **distinct** location c[i] and reads **distinct** locations a[i], b[i]. No two threads ever touch the same memory element, so there is no data race — no lock is needed. At the end of the parallel for, OpenMP inserts an **implicit barrier**: no thread leaves the loop until all threads have finished their iterations. This is what guarantees c[] is fully populated before the verification loop runs.

5. By default, how are the iterations divided among the threads (think of the static schedule)? Would you expect the division to matter for this loop?

By default OpenMP uses a **static schedule** — the iteration space is divided into contiguous chunks of roughly N/t iterations each, one chunk per thread (assigned at the start). For N=5,000,000 and 4 threads, each thread gets ~1,250,000 iterations.

For this loop the division **does not matter** because every iteration performs identical work (same number of adds and memory accesses). There is no load imbalance to correct. Scheduling would matter for loops where per-iteration cost varies significantly — e.g., conditional branches, triangular loops, or work based on data.

LAB TASK 03

Code

Task3 > C scoping_bug.c > ...

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000

int main(void)
{
    int n = N;   /* variable, not macro - used in shared() */
    int *in  = malloc(n * sizeof(int));
    int *out = malloc(n * sizeof(int));

    if (!in || !out) { fprintf(stderr, "malloc failed\n"); return 1; }

    for (int i = 0; i < n; i++) in[i] = i % 100;

    int tmp;   /* shared by default - THE BUG */

#ifdef FIX_PRIVATE
    #pragma omp parallel for default(none) shared(in, out, n) private(tmp)
#elif defined(FIX_LOCAL)
    #pragma omp parallel for default(none) shared(in, out, n)
#elif defined(DEFAULT_NONE)
    #pragma omp parallel for default(none) shared(in, out, n)
#else
    #pragma omp parallel for
#endif
    for (int i = 0; i < n; i++)
    {
#ifdef FIX_LOCAL
        int tmp = in[i] * 3;
#else
```

```
30      #ifdef FIX_LOCAL
31      |       int tmp = in[i] * 3;
32      #else
33      |       tmp = in[i] * 3;
34      #endif
35      |       out[i] = tmp + 7;
36      |   }
37      |
38      |   int wrong = 0;
39      |   for (int i = 0; i < n; i++)
40      |   {
41      |       int expected = (in[i] * 3) + 7;
42      |       if (out[i] != expected) wrong++;
43      |   }
44      |
45      |   printf("Wrong elements: %d\n", wrong);
46      |
47      |   free(in); free(out);
48      |   return 0;
49      }
50
```

Output

1.

```
#define N 1000000
PS E:\Lab-06\Task3> gcc -O0 -fopenmp -std=gnu11 scoping_bug.c -o bug.exe
PS E:\Lab-06\Task3> gcc -O0 -fopenmp -std=gnu11 -DFIX_PRIVATE scoping_bug.c
-o fix_private.exe
PS E:\Lab-06\Task3> gcc -O0 -fopenmp -std=gnu11 -DFIX_LOCAL   scoping_bug.c
-o fix_local.exe
PS E:\Lab-06\Task3> gcc -O0 -fopenmp -std=gnu11 -DDEFAULT_NONE scoping_bug
 -o fix_none.exe
scoping_bug.c: In function 'main':
scoping_bug.c:33:13: error: 'tmp' not specified in enclosing 'parallel'
   33 |         tmp = in[i] * 3;
      |         ~~~~^~~~~~~~~~
scoping_bug.c:24:13: note: enclosing 'parallel'
   24 |     #pragma omp parallel for default(none) shared(in, out, n)
      |             ^~~
```

2.

```
PS E:\Lab-06\Task3> Write-Host "===== Buggy, 4 threads, 10 runs ====="
===== Buggy, 4 threads, 10 runs =====
PS E:\Lab-06\Task3> $env:OMP_NUM_THREADS=4
PS E:\Lab-06\Task3> 1..10 | ForEach-Object { Write-Host -NoNewline "[$_] ";
.\bug.exe }
[1] Wrong elements: 24287
[2] Wrong elements: 10399
[3] Wrong elements: 77427
```

```
[1] Wrong elements: 24287
[2] Wrong elements: 10399
[3] Wrong elements: 77427
[4] Wrong elements: 35471
[5] Wrong elements: 9657
[6] Wrong elements: 14669
[7] Wrong elements: 19340
[8] Wrong elements: 6854
[9] Wrong elements: 43821
[10] Wrong elements: 52574
PS E:\Lab-06\Task3>
PS E:\Lab-06\Task3> Write-Host "===== Buggy, 1 thread ====="
===== Buggy, 1 thread =====
PS E:\Lab-06\Task3> $env:OMP_NUM_THREADS=1; .\bug.exe
Wrong elements: 0
PS E:\Lab-06\Task3>
PS E:\Lab-06\Task3> Write-Host "===== FIX_PRIVATE, 4 threads, 10 runs ====="

===== FIX_PRIVATE, 4 threads, 10 runs =====
PS E:\Lab-06\Task3> $env:OMP_NUM_THREADS=4
PS E:\Lab-06\Task3> 1..10 | ForEach-Object { Write-Host -NoNewline "[$_] "; .\fix_private.e
xe }
[1] Wrong elements: 0
[2] Wrong elements: 0
[3] Wrong elements: 0
[4] Wrong elements: 0
[5] Wrong elements: 0
[6] Wrong elements: 0
[7] Wrong elements: 0
[8] Wrong elements: 0
[9] Wrong elements: 0
[10] Wrong elements: 0
PS E:\Lab-06\Task3>
PS E:\Lab-06\Task3> Write-Host "===== FIX_LOCAL, 4 threads, 10 runs ====="
===== FIX_LOCAL, 4 threads, 10 runs =====
PS E:\Lab-06\Task3> 1..10 | ForEach-Object { Write-Host -NoNewline "[$_] "; .\fix_local.exe
 }
[1] Wrong elements: 0
[2] Wrong elements: 0
[3] Wrong elements: 0
[4] Wrong elements: 0
[5] Wrong elements: 0
[6] Wrong elements: 0
[7] Wrong elements: 0
[8] Wrong elements: 0
[9] Wrong elements: 0
[10] Wrong elements: 0
PS E:\Lab-06\Task3>
```

Result

Run	Buggy	FIX_PRIVATE	FIX_LOCAL
1	24287	0	0
2	10399	0	0
3	77427	0	0
4	35471	0	0
5	9657	0	0
6	14669	0	0
7	19340	0	0
8	6854	0	0
9	43821	0	0
10	52574	0	0

Buggy with OMP_NUM_THREADS=1, **wrong elements: 0 nproc: 4**

-DDEFAULT_NONE **compile error message:**

text

error: 'tmp' not specified in enclosing 'parallel'

Discussion Questions

1. Did the buggy version give the same number of wrong elements in every run, and what did it print with one thread? Why is that behaviour typical of a race condition, and why is a correct-looking run no proof of correctness?

No — the buggy version gave a **different** number of wrong elements in every run (24287, 10399, 77427, 35471, 9657, 14669, 19340, 6854, 43821, 52574). With **1 thread**, it printed **0 wrong elements** because there is no concurrency to race against. This is the classic signature of a race condition: non-deterministic output, timing-dependent. A correct-looking single-thread run proves nothing — the bug is still in the code and will manifest on other runs, other machines, or other thread counts.

2. Using two threads and the statements tmp = in[i] * 3; **and** out[i] = tmp + 7;**, write out an interleaving that makes** out[i] **wrong.**

Two threads T1 and T2, tmp is shared. Suppose T1 handles i=10 and T2 handles i=500:

Step	T1 (i=10)	T2 (i=500)	tmp
1	tmp = in[10]*3 → tmp = 30		30
2		tmp = in[500]*3 → tmp = 1500	1500
3	out[10] = tmp + 7 → **1507** (WRONG, should be 37)		1500
4		out[500] = tmp + 7 → 1507	1500

T1 read tmp **after** T2 had overwritten it → T1's result is wrong.

3. Compare private**,** firstprivate **and** lastprivate**: what value does a private copy start with, and which clause would you use if the copy had to start with the value of the original variable?**

Clause	Initial value of the copy	Copies back to original?
private(x)	**Uninitialized** (garbage)	No
firstprivate(x)	**Copy of the original's value**	No
lastprivate(x)	Uninitialized	Yes — last iteration's value

Use firstprivate if the private copy must start with the value the original had **before** the parallel region.

4. What does default(none) **do, and why is a compile error preferable to a wrong result at run time?**

default(none) disables all implicit data-sharing — every variable used inside the parallel region must be explicitly declared as shared, private, firstprivate, or lastprivate. If any variable is left out, the compiler errors out. A compile error is preferable because it catches the mistake **before the program ships**. A silent wrong result at run time may only appear on certain machines, under certain loads, and is much harder to debug.

5. Why is the buggy build compiled with -O0**? Would a correct-looking** -O2 **result mean the bug was fixed?**

The buggy build uses -O0 because optimizations can hide or change the race — the compiler may keep tmp in a register per thread (treating it as if it were private), or reorder statements in a way that makes the race disappear. A -O2 run that gives correct output does **not** mean the bug is **fixed**. The code still has undefined behaviour (a data race). Different compilers, optimization levels, or CPUs could expose it. The correct fix is private(tmp) or declaring tmp inside the loop.

LAB TASK 04

Code

Task4 > C omp_sections.c > ...

```c
#include <stdio.h>
#include <omp.h>

static void workA(void) { volatile double s = 0; for (long i = 0; i < 300000000L; i++) s += i; }
static void workB(void) { volatile double s = 0; for (long i = 0; i < 200000000L; i++) s += i; }
static void workC(void) { volatile double s = 0; for (long i = 0; i < 100000000L; i++) s += i; }

int main(void)
{
    double t0 = omp_get_wtime();

    #pragma omp parallel sections
    {
        #pragma omp section
        {
            int tid = omp_get_thread_num();
            printf("Section A executed by thread %d\n", tid);
            workA();
        }

        #pragma omp section
        {
            int tid = omp_get_thread_num();
            printf("Section B executed by thread %d\n", tid);
            workB();
        }

        #pragma omp section
        {
            int tid = omp_get_thread_num();
            printf("Section C executed by thread %d\n", tid);
            workC();
        }
    }

    double t1 = omp_get_wtime();
    printf("Wall time = %.3f s\n", t1 - t0);
    return 0;
}

```

Output

```
PS E:\Lab-06\Task4> foreach ($t in 1,2,3,4,8) {
>>     Write-Host "===== OMP_NUM_THREADS=$t ====="
>>     $env:OMP_NUM_THREADS=$t; .\omp_sections.exe
>> }
===== OMP_NUM_THREADS=1 =====
Section A executed by thread 0
Section B executed by thread 0
Section C executed by thread 0
Wall time = 2.093 s
===== OMP_NUM_THREADS=2 =====
Section A executed by thread 0
Section B executed by thread 1
Section C executed by thread 1
Wall time = 1.091 s
===== OMP_NUM_THREADS=3 =====
Section A executed by thread 1
Section B executed by thread 0
Section C executed by thread 2
Wall time = 1.162 s
===== OMP_NUM_THREADS=4 =====
Section C executed by thread 0
Section A executed by thread 2
Section B executed by thread 3
Wall time = 1.092 s
===== OMP_NUM_THREADS=8 =====
Section B executed by thread 1
Section A executed by thread 0
Section C executed by thread 3
Wall time = 0.974 s
PS E:\Lab-06\Task4>
PS E:\Lab-06\Task4> Write-Host "===== 3 threads, 3 repeated runs ====="
===== 3 threads, 3 repeated runs =====
PS E:\Lab-06\Task4> $env:OMP_NUM_THREADS=3
PS E:\Lab-06\Task4> 1..3 | ForEach-Object { Write-Host "[$_]"; .\omp_sections.exe }
[1]
Section A executed by thread 1
Section B executed by thread 0
Section C executed by thread 2
Wall time = 0.861 s
[2]
Section B executed by thread 1
Section A executed by thread 0
Section C executed by thread 2
Wall time = 1.069 s
[3]
Section B executed by thread 2
Section A executed by thread 0
Section C executed by thread 1
Wall time = 1.338 s
PS E:\Lab-06\Task4>
```

Discussion Questions

Threads	Wall time (s)	Speedup	Thread of A / B / C	Results match?
1	2.161	**1.00×**	0 / 0 / 0	Y
2	1.216	**1.78×**	0 / 1 / 1	Y
3	1.278	**1.69×**	1 / 0 / 2	Y
4	1.252	**1.73×**	2 / 0 / 1	Y
8	1.195	**1.81×**	0 / 1 / 3	Y

Speedup formula: Speedup(t) = Time(1 thread) / Time(t threads)

- t=2: 2.161 / 1.216 = **1.78×**
- t=3: 2.161 / 1.278 = **1.69×**
- t=4: 2.161 / 1.252 = **1.73×**
- t=8: 2.161 / 1.195 = **1.81×**

SERIAL TIMES

Aapke 1-thread run ka wall time = **A + B + C** = 2.161 s.

Loop counts: A = 300M, B = 200M, C = 100M → ratio **3 : 2 : 1**. Total 6 units.

Section	Loop count	Estimated serial time (s)
A	300,000,000	**1.081**
B	200,000,000	**0.720**
C	100,000,000	**0.360**
Total	600,000,000	**2.161**

Longest section (critical path): A = **1.081 s**

Theoretical best wall time with 3+ threads: ~1.081 s (limited by A)

Actual with 3 threads: 1.278 s — close to A, as expected.

02456

DISCUSSION ANSWERS

1. Which operation took longest? Compare the wall time with 3 threads against the longest single section and against the serial total, and explain what limits the speedup.

Section A took longest (300M iterations → about **1.081 s** serial). The wall time with 3 threads was **1.278 s** — much closer to the longest single section (1.081 s) than to the serial total (2.161 s). This is expected: in sections, all three sections run **in parallel**, so the wall time is dominated by the **slowest section** (the critical path). The speedup is limited by the longest section, not by the total work. This is a direct demonstration of **Amdahl's law** — the speedup is bounded by the part that cannot be parallelised further.

2. Why did 4 and 8 threads not improve on 3 threads, and what happened to the extra threads?

There are only **3 sections**, so at most 3 threads can do useful work at once. The 4th–8th threads are **idle** — OpenMP has no work to give them. Wall times confirm this: 3 threads = 1.278 s, 4 threads = 1.252 s, 8 threads = 1.195 s — differences are within noise; there is no meaningful improvement. The extra threads sit at the implicit barrier at the end of the sections construct, waiting for the 3 working threads to finish.

3. Was the same thread assigned to the same section in every run? Does the OpenMP standard guarantee any particular assignment?

No — assignment varied between runs:

- 3-thread run [1]: A→0, B→2, C→1
- 3-thread run [2]: A→0, B→2, C→1
- 3-thread run [3]: A→2, B→0, C→1
- Earlier 3-thread run: A→1, B→0, C→2

The OpenMP standard **does not guarantee** which thread executes which section. The runtime assigns whichever thread is free when a section starts. Therefore, code must **never assume** that a particular thread runs a particular section.

4. Why must sections be independent? Describe what could go wrong if sections A and B wrote to the same variable without synchronization.

Sections run **concurrently** on different threads, so if they share mutable data, we get a **data race** — exactly the kind seen in Task 3. For example:

2026

```
c

// Section A

x = x + 1;


// Section B

x = x + 1;
```

Both sections could load the same x, both add 1, and one store overwrites the other → a **lost update**. Similarly, if A reads data that B is still writing, A may see a **stale or torn** value. Sections must either use **disjoint** data, or protect shared accesses with critical / atomic / barrier.

5. When would you choose sections rather than parallel for, and what does the implicit barrier at the end of the construct mean for the code after it?

Use sections when you have a **small number of different, independent tasks** (e.g., 3–4 tasks): different kernels, different data structures, different phases of a pipeline. Use parallel for when you have **one loop with many iterations** that can be split evenly.

The **implicit barrier** at the end of sections means that no thread leaves the construct until **every section has completed**. So the code immediately after the parallel block is guaranteed to see the **final, consistent** results of all sections. This is what makes the verification after the block safe. Adding nowait would remove that guarantee.

2026